

Lesson 17: Multi-Environment

Learning Objectives

- Structure Terraform configurations for multiple environments
 - Use directory layout and variable files for dev, qa, and prod
 - Use workspaces as an alternative to directory-per-environment
 - Understand when to use each approach
-

1. Why Multi-Environment?

Real-world infrastructure needs separate environments:

```
dev      → Development and experimentation
qa       → Testing and validation
staging  → Pre-production mirror
prod     → Live customer traffic
```

Each environment typically has:

- Different resource names (e.g., `myapp-dev` vs `myapp-prod`)
 - Different sizes/tiers (small instances in dev, large in prod)
 - Different configurations (versioning on in dev, off in prod)
 - Different access controls
-

2. Directory Structure Approach

The most common pattern — separate directories per environment, sharing modules:

```
terraform/
├── modules/
│   ├── s3-bucket/
│   ├── sqs-queue/
│   └── dynamodb-table/
├── dev/
│   ├── main.tf
│   ├── provider.tf
│   ├── variables.tf
│   ├── terraform.tfvars
│   └── outputs.tf
├── qa/
│   ├── main.tf
│   └── provider.tf
```

```
├── variables.tf
├── terraform.tfvars
├── outputs.tf
└── prod/
    ├── main.tf
    ├── provider.tf
    ├── variables.tf
    ├── terraform.tfvars
    └── outputs.tf
```

Shared Modules

All environments call the same modules:

```
# dev/main.tf – same as qa/main.tf and prod/main.tf
module "bucket" {
  source = "../modules/s3-bucket"

  bucket_name      = var.bucket_name
  enable_versioning = var.enable_versioning
  tags             = var.tags
}

module "queue" {
  source = "../modules/sqs-queue"

  queue_name      = var.queue_name
  delay_seconds   = var.delay_seconds
  tags            = var.tags
}
```

Environment-Specific tfvars

```
# dev/terraform.tfvars
environment      = "dev"
bucket_name      = "myapp-dev-data"
enable_versioning = true
delay_seconds    = 5
tags = {
  Environment = "dev"
  Project     = "myapp"
}
```

```
# prod/terraform.tfvars
environment      = "prod"
bucket_name      = "myapp-prod-data"
```

```
enable_versioning = false
delay_seconds     = 0
tags = {
  Environment = "prod"
  Project     = "myapp"
}
```

Deployment Commands

```
# Deploy dev
cd dev
terraform init
terraform apply

# Deploy prod
cd ../prod
terraform init
terraform apply
```

3. Single Directory with Var Files

A simpler approach — one directory with multiple `.tfvars` files:

```
terraform/
├── main.tf
├── provider.tf
├── variables.tf
├── outputs.tf
├── modules/
│   ├── s3-bucket/
│   └── sqs-queue/
├── dev.tfvars
├── qa.tfvars
└── prod.tfvars
```

Deployment Commands

```
# Deploy dev
terraform apply -var-file="dev.tfvars"

# Deploy prod
terraform apply -var-file="prod.tfvars"
```

Environment Variable Override

```
# Override specific values
terraform apply -var-file="dev.tfvars" -var="enable_versioning=false"
```

4. Workspaces Approach

Terraform workspaces provide named state files within the same directory:

```
# Create workspaces
terraform workspace new dev
terraform workspace new qa
terraform workspace new prod

# Switch workspace
terraform workspace select dev
terraform workspace select prod

# List workspaces
terraform workspace list

# Show current
terraform workspace show
```

Using Workspace in Configuration

```
locals {
  environment = terraform.workspace
  is_prod     = terraform.workspace == "prod"

  bucket_name = "myapp-${terraform.workspace}-data"

  enable_versioning = !local.is_prod
  delay_seconds     = local.is_prod ? 0 : 5
}

resource "aws_s3_bucket" "data" {
  bucket = local.bucket_name
  tags = {
    Environment = terraform.workspace
  }
}
```

Workspace State Storage

Workspaces use separate state files:

```
terraform.tfstate.d/
├── dev/
│   └── terraform.tfstate
├── qa/
│   └── terraform.tfstate
└── prod/
    └── terraform.tfstate
```

5. Approach Comparison

Aspect	Directory-per-env	Var files	Workspaces
Separation	Full isolation	Shared code, different vars	Shared code, different state
State isolation	Separate directories	Same directory, manual select	Automatic separate states
Complexity	More files	Minimal	Medium
Module path	<code>../modules/</code>	<code>./modules/</code>	<code>./modules/</code>
Best for	Large teams, compliance	Small teams, few envs	Single team, quick switching

6. Complete Example (Var Files Approach)

variables.tf

```
variable "environment" {
  description = "Deployment environment"
  type        = string
}

variable "bucket_name" {
  description = "S3 bucket name"
  type        = string
}

variable "queue_name" {
  description = "SQS queue name"
  type        = string
}

variable "enable_versioning" {
  description = "Enable S3 versioning"
  type        = bool
  default     = true
}
```

```
variable "tags" {  
  description = "Resource tags"  
  type        = map(string)  
  default     = {}  
}
```

dev.tfvars

```
environment      = "dev"  
bucket_name      = "myapp-dev-data"  
queue_name       = "myapp-dev-events"  
enable_versioning = true  
tags = {  
  Environment = "dev"  
  Project     = "myapp"  
}
```

prod.tfvars

```
environment      = "prod"  
bucket_name      = "myapp-prod-data"  
queue_name       = "myapp-prod-events"  
enable_versioning = false  
tags = {  
  Environment = "prod"  
  Project     = "myapp"  
}
```

7. Key Takeaways

- Multi-environment is managed via **directories**, **var files**, or **workspaces**
- Directory-per-env gives full isolation but more duplication
- Single directory with var files is simplest for small teams
- Workspaces are built-in Terraform state isolation
- Use `terraform.workspace` in configs to reference the current workspace
- Always pin modules — environments should use the same module version
- Never hardcode environment-specific values in modules — pass them via variables

example and exercise repo url

<https://github.com/eyu-se/terraform-training-material-with-projects>

Trainer - Eyuel M.